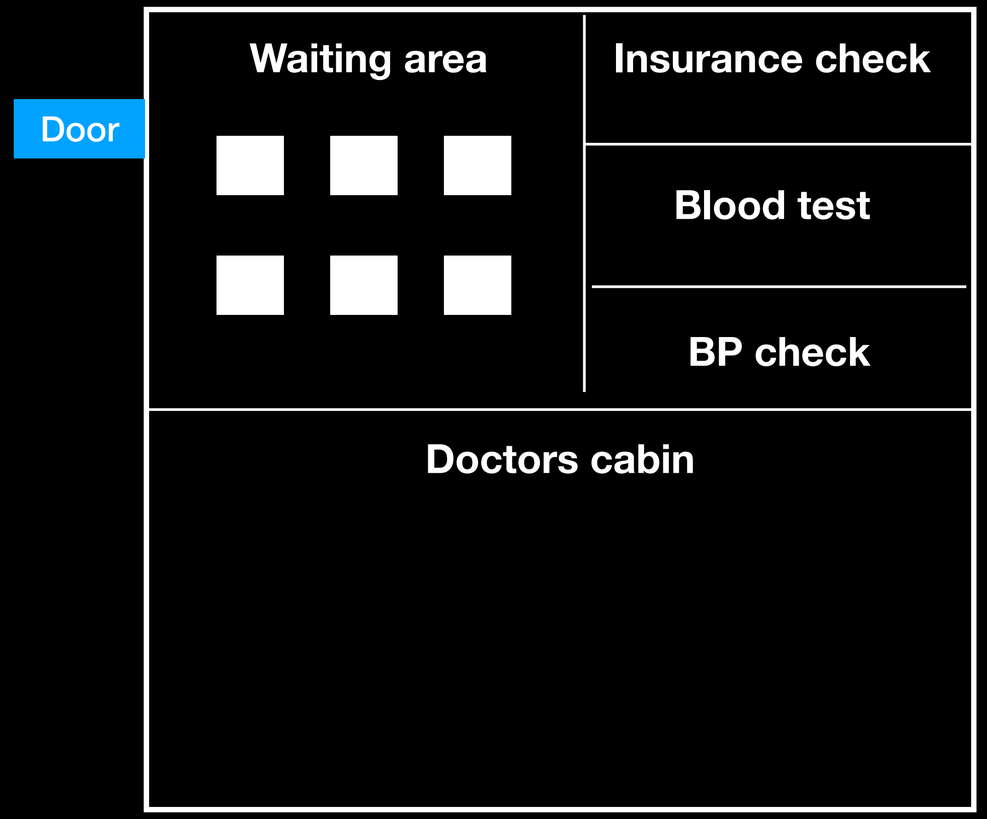
# 3.1
# Middlewares, authentication, global catches, zod

# Hospital (covid)

Before  you reach the doctor

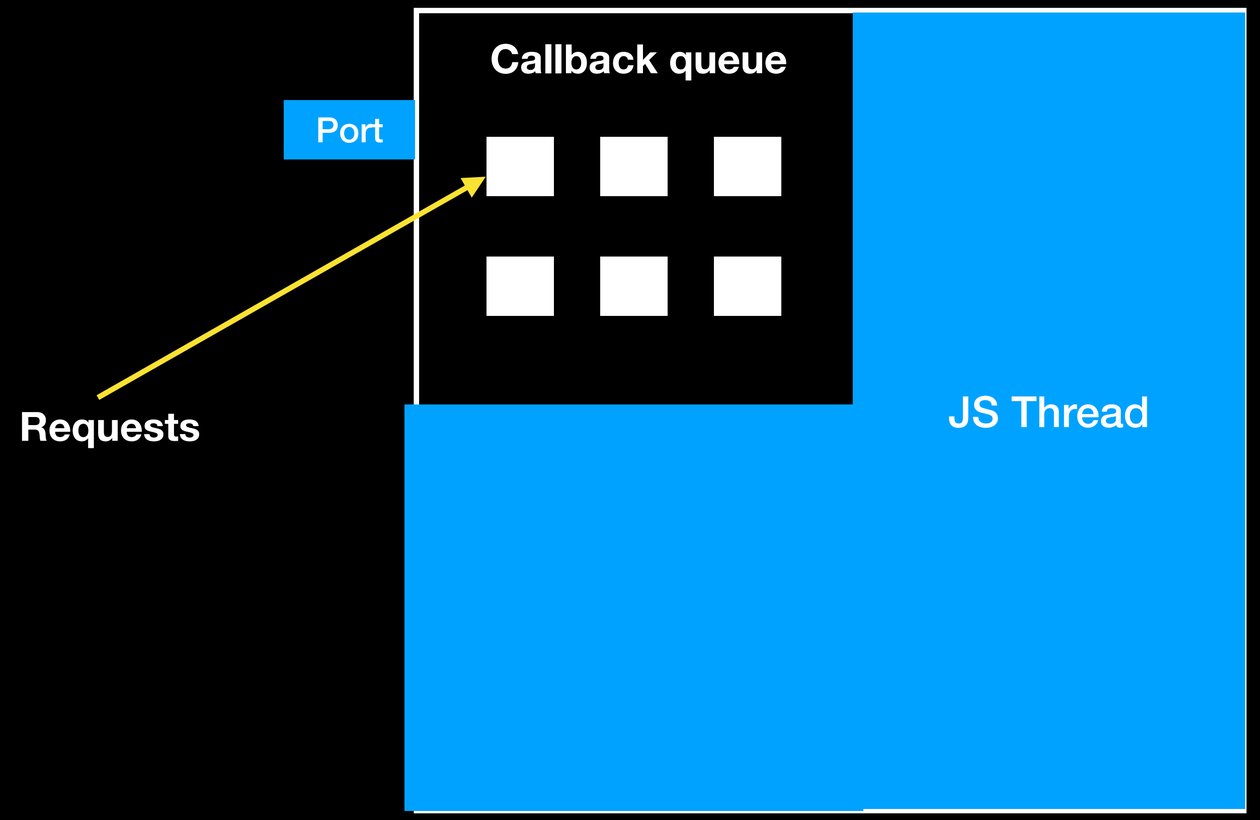
1. Your adhar/insurance info is taken. Only if you have insurance you proceed
2. Blood test is done, only if no STD does use proceed
3. BP is checked, only if BP is reasonable user proceeds



# Hospital (covid)

Before  you reach the doctor

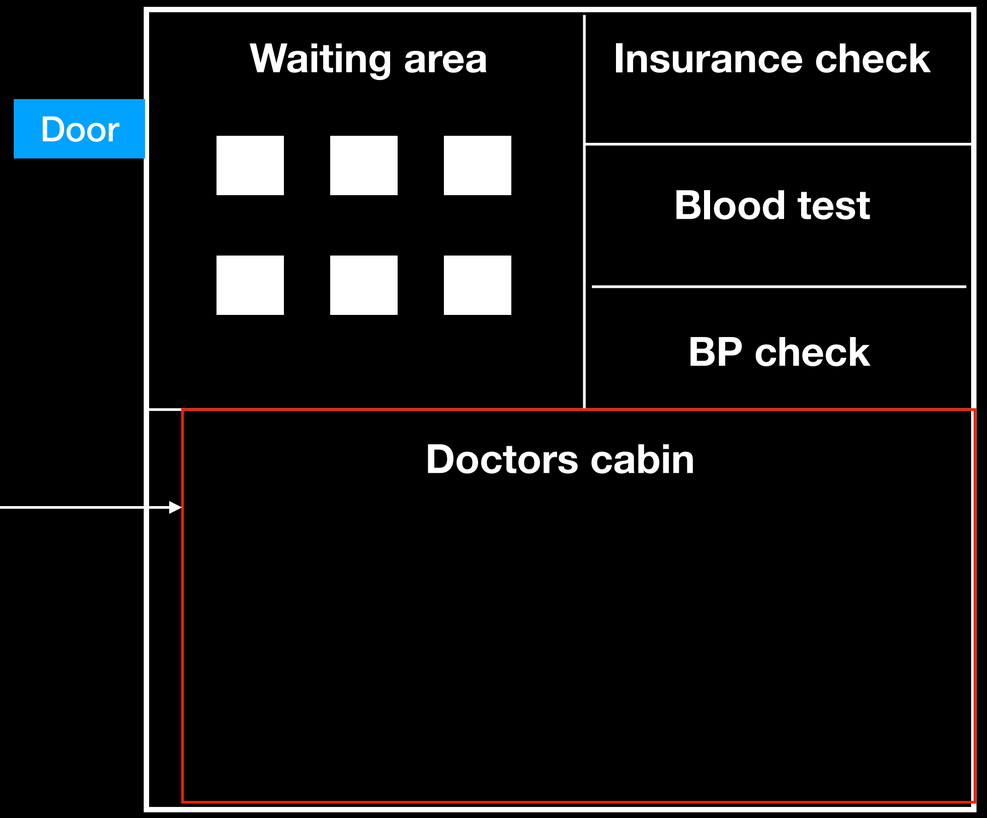
1. Your adhar/insurance info is taken
2. Blood test is done
3. BP is checked



# Equivalent code

index.js > ...

```
2  const express = require("express");
3
4  const app = express();
5
6  app.get("/health-checkup", function (req, res) {
7    // do health checks here
8    res.send("Your heart is healthy");
9  });
10
```



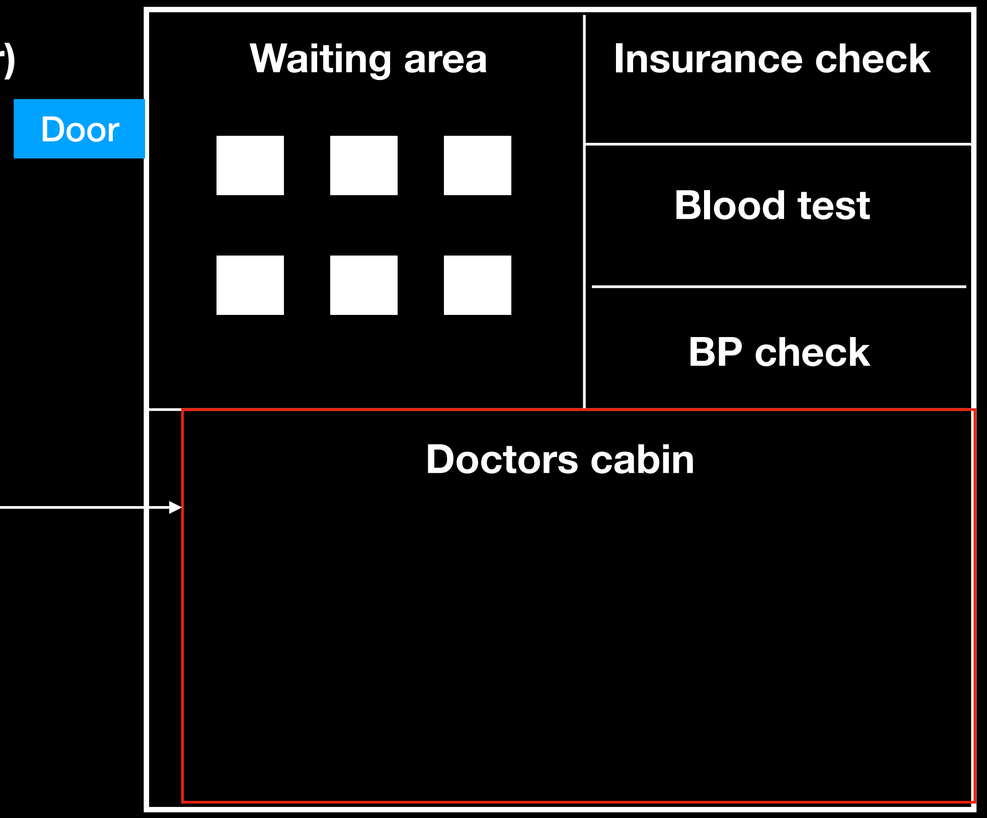
# Questions

How do you do

1. Auth checks? (Does this user have funds to visit the doctor)
2. Ensure input by the user is valid (BP / blood tests)

index.js > ...

```
2  const express = require("express");
3
4  const app = express();
5
6  app.get("/health-checkup", function (req, res) {
7    // do health checks here
8    res.send("Your heart is healthy");
9  });
10
```



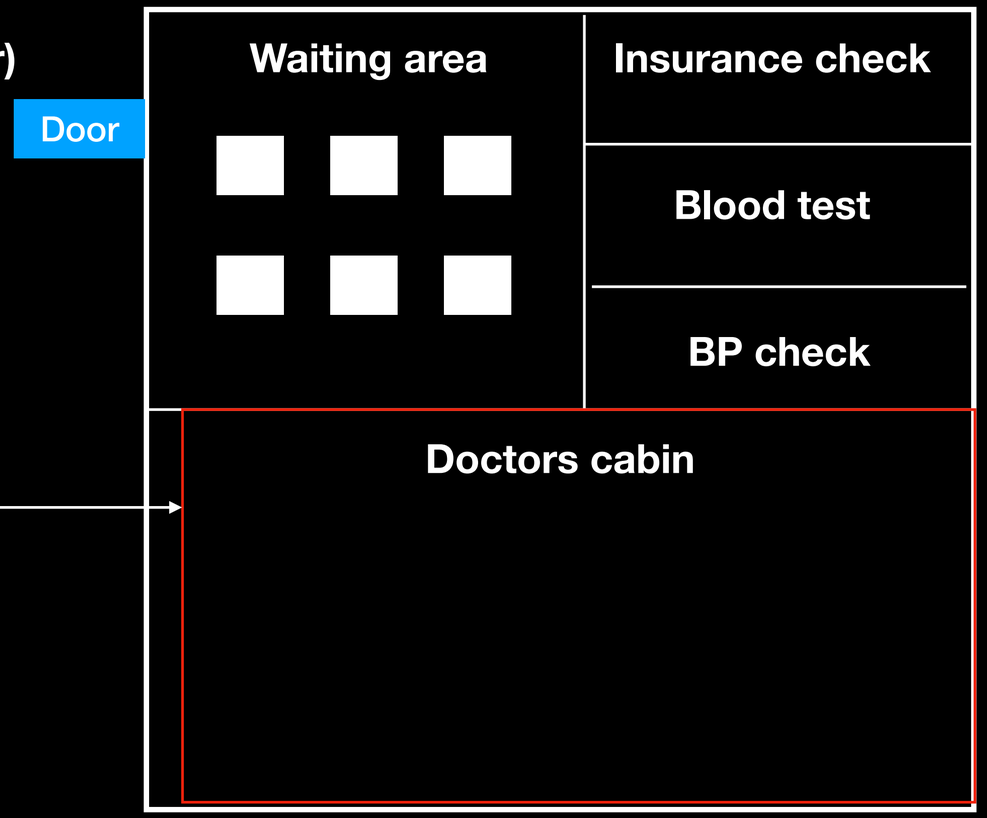
# Answer - Middlewares

How do you do

1. Auth checks? (Does this user have funds to visit the doctor)
2. Ensure input by the user is valid (BP / blood tests)

index.js > ...

```
2  const express = require("express");
3
4  const app = express();
5
6  app.get("/health-checkup", function (req, res) {
7    // do health checks here
8    res.send("Your heart is healthy");
9  });
10
```



# Answer - Middlewares

**Before we proceed, lets add constraints to our route**

1. **User needs to send a kidneyId as a query param which should be a number from 1-2 (humans only has 2 kidneys)**
2. **User should send a username and password in headers**

index.js > ...

```
const express = require("express");

const app = express();

app.get("/health-checkup", function (req, res) {
  // do health checks here
  res.send("Your heart is healthy");
});
```

# Answer - Middlewares

**Before we proceed, lets add constraints to our route**

1. **User needs to send a kidneyId as a query param which should be a number from 1-2 (humans only has 2 kidneys)**
2. **User should send a username and password in headers**

index.js > ...

```
const express = require("express");

const app = express();

app.get("/health-checkup", function (req, res) {
  // do health checks here
  res.send("Your heart is healthy");
});
```

# Answer - Middlewares

**Before we proceed, lets add constraints to our route**

1. **User needs to send a kidneyId as a query param which should be a number from 1-2 (humans only has 2 kidneys)**
2. **User should send a username and password in headers**

**Username checks**

**Input validation**

index.js > app.get("/health-checkup") callback > ...

```
2  const express = require("express");
3
4  const app = express();
5
6  app.get("/health-checkup", function (req, res) {
7    // do health checks here
8    const kidneyId = req.query.kidneyId;
9    const username = req.headers.username;
10   const password = req.headers.password;
11
12   if (username != "harkirat" && password != "pass") {
13     res.status(403).json({
14       msg: "User doesnt exist",
15     });
16     return;
17   }
18
19   if (kidneyId != 1 && kidneyId != 2) {
20     res.status(411).json({
21       msg: "wrong inputs",
22     });
23     return;
24   }
25   // do something with kidney here
26
27   res.send("Your heart is healthy");
28 });
29
```

# Answer - Middlewares

Before we proceed, lets add constraints to our route

1. User needs to send a kidneyId as a query param which should be a number from 1-2 (humans only has 2 kidneys)
2. User should send a username and password in headers

What if I tell you to introduce another route that does
Kidney replacement
Inputs need to be the same

Username checks

Input validation

index.js > app.get("/health-checkup") callback > ...

```
2  const express = require("express");
3
4  const app = express();
5
6  app.get("/health-checkup", function (req, res) {
7    // do health checks here
8    const kidneyId = req.query.kidneyId;
9    const username = req.headers.username;
10   const password = req.headers.password;
11
12   if (username != "harkirat" && password != "pass") {
13     res.status(403).json({
14       msg: "User doesnt exist",
15     });
16     return;
17   }
18
19   if (kidneyId != 1 && kidneyId != 2) {
20     res.status(411).json({
21       msg: "wrong inputs",
22     });
23     return;
24   }
25   // do something with kidney here
26
27   res.send("Your heart is healthy");
28 });
29
```

# Middlewares

**What if I tell you to introduce another route that does Kidney replacement**
**Inputs need to be the same**

**Ugly solution - Create a new route, repeat code**

index.js > app.put("/replace-kidney") callback > ...

```javascript
const express = require("express");

const app = express();

app.get("/health-checkup", function (req, res) {
  // do health checks here
  const kidneyId = req.query.kidneyId;
  const username = req.headers.username;
  const password = req.headers.password;

  if (username != "harkirat" && password != "pass") {
    res.status(403).json({
      msg: "User doesnt exist",
    });
    return;
  }

  if (kidneyId != 1 && kidneyId != 2) {
    res.status(411).json({
      msg: "wrong inputs",
    });
    return;
  }
  // do something with kidney here

  res.send("Your heart is healthy");
});

app.put("/replace-kidney", function (req, res) {
  // do health checks here
  const kidneyId = req.query.kidneyId;
  const username = req.headers.username;
  const password = req.headers.password;

  if (username != "harkirat" && password != "pass") {
    res.status(403).json({
      msg: "User doesnt exist",
    });
    return;
  }

  if (kidneyId != 1 && kidneyId != 2) {
    res.status(411).json({
      msg: "wrong inputs",
    });
    return;
  }
  // do kidney replacement logic here

  res.send("Your heart is healthy");
});
```

# Middlewares

What if I tell you to introduce another route that does
Kidney replacement
Inputs need to be the same

Slightly better solution - Create wrapper fns

```
function usernameValidator(username, password) {
  if (username != "harkirat" && password != "pass") {
    return false;
  }
  return true
}

function kidneyValidator(kidneyId) {
  if (kidneyId != 1 && kidneyId != 2) {
    return false;
  }
  return true;
}

app.get("/health-checkup", function (req, res) {
  // do health checks here
  const kidneyId = req.query.kidneyId;

  if (!usernameValidator(req.query.username, req.query.password)) {
    res.status(403).json({
      msg: "User doesnt exist",
    });
    return;
  }

  if (!kidneyValidator(kidneyId)) {
    res.status(411).json({
      msg: "wrong inputs",
    });
    return;
  }
  // do something with kidney here

  res.send("Your heart is healthy");
});

app.put("/replace-kidney", function (req, res) {
  // do health checks here
  const kidneyId = req.query.kidneyId;
  const username = req.headers.username;
  const password = req.headers.password;

  if (!usernameValidator(req.query.username, req.query.password)) {
    res.status(403).json({
      msg: "User doesnt exist",
    });
    return;
  }

  if (!kidneyValidator(kidneyId)) {
    res.status(411).json({
      msg: "wrong inputs",
    });
    return;
  }
  // do kidney replacement logic here

  res.send("Your heart is healthy");
```

# Middlewares

Defining middleware (just another fn)

Using the middleware

```
index.js > app.get("/heart-check") callback
4   const app = express();
5
6   function userMiddleware(req, res, next) {
7     if (username != "harkirat" && password != "pass") {
8       res.status(403).json({
9         msg: "Incorrect inputs",
10      });
11    } else {
12      next();
13    }
14  };
15
16  function kidneyMiddleware(req, res, next) {
17    if (kidneyId != 1 && kidneyId != 2) {
18      res.status(403).json({
19        msg: "Incorrect inputs",
20      });
21    } else {
22      next();
23    }
24  };
25
26  app.get("/health-checkup", userMiddleware, kidneyMiddleware, function (req, res) {
27    // do something with kidney here
28
29    res.send("Your heart is healthy");
30  });
31
32
33  app.get("/kidney-check", userMiddleware, kidneyMiddleware, function (req, res) {
34    // do something with kidney here
35
36    res.send("Your heart is healthy");
37  });
38
39  app.get("/heart-check", userMiddleware, function (req, res) {
40    // do something with user here
41
42    res.send("Your heart is healthy");
43  });
44
```

What if I tell you to introduce another route that does Kidney replacement
Inputs need to be the same

Best solution - middleware

Last thing in middleware - app.use

```
const app = express();
app.use(express.json());
```

Other use cases of middleware (assignment) =

1. Count the number of requests
2. Find the average time your server is taking to handle requests

Take a breather
We understand middlewares

# Why do you need input validation?

## Lets see with an example

What if the user sends the wrong body?

index.js > ...

```javascript
const express = require("express");

const app = express();

app.post("/health-checkup", function (req, res) {
  // do something with kidney here
  const kidneys = req.body.kidneys;
  const kidneyLength = kidneys.length;

  res.send("Your kidney length is " + kidneyLength);
});

app.listen(3000);
```

# app.use(express.json())

The line app.use(express.json()) in your Express application is used to parse incoming requests with JSON payloads. This middleware function is part of Express 4.16.0 and later, and it helps you handle JSON data sent in the request body.

Purpose of app.use(express.json())
When you use app.use(express.json()), it tells the Express app to use the JSON middleware. This middleware function parses incoming JSON requests and puts the parsed data in req.body. Without this middleware, req.body would be undefined when you try to access the JSON data from the client.
For example, in your code:

```
app.post("/health-checkup", (req, res) => {
    const kidneys = req.body.kidneys;
    const kidneysLength = kidneys.length;

    res.send("You have " + kidneysLength + " kidneys.");
});
```

req.body.kidneys would be undefined if you didn't use express.json() because the server wouldn't know how to parse the JSON data in the request.

Difference between app.use(express.json()) and app.use(bodyParser.json())

Before Express 4.16.0, to parse JSON data, you would typically use the body-parser middleware. The body-parser library provides various middleware to handle different types of request bodies, including bodyParser.json() for JSON payloads.

Here’s a comparison: app.use(express.json()):

Introduced in Express 4.16.0.
Functionally equivalent to bodyParser.json().
No need to install an additional package (body-parser), as it’s included in Express.

app.use(bodyParser.json()):

Used in older versions of Express or for more complex configurations.
Requires installing the body-parser package separately.
Allows for more detailed configuration if you need it (e.g., setting limits on request body size).

Here is how you would use body-parser:

```
const bodyParser = require('body-parser');
app.use(bodyParser.json());
```

Summary

Use app.use(express.json()) if you are using Express 4.16.0 or later. It’s simpler and doesn’t require an additional package.

Use app.use(bodyParser.json()) if you are using an older version of Express or if you need the additional configuration options provided by body-parser.

Both methods serve the same purpose: parsing incoming requests with JSON payloads and populating req.body with the parsed data.

# Why do you need input validation?
# Lets see with an example

**Global catches** help you give the user a
Better error message

**Error-Handling Middleware**: This is a special type of middleware function in Express that has four arguments instead of three (`(err, req, res, next)`). Express recognizes it as an error-handling middleware because of these four arguments.

index.js > app.post("/health-checkup") callback

```javascript
const express = require("express");

const app = express();

app.post("/health-checkup", function (req, res) {
  // do something with kidney here
  const kidneys = req.body.kidneys;
  const kidneyLength = kidneys.length;

  res.send("Your kidney length is " + kidneyLength);
});

app.use((error, req, res, next) => {
    // console.error(error); // Log the error for debugging
    res.status(500).send('An internal server error occurred');
});

app.listen(3000);
```

# 3.1
# Middlewares, authentication, global catches, zod

# How can you do better input validation?

```
if (kidneyId != 1 && kidneyId != 2) {
  return false;
}
```

This is very hard to scale
What if you expect a complicated input?

# How can you do better input validation?

```
if (kidneyId != 1 && kidneyId != 2) {
  return false;
}
```

This is where zod comes into the picture

# How can you do better input validation?

This is where zod comes into the picture

index.js > z

```javascript
const express = require("express");
const z = require("zod");
const app = express();

app.use(express.json())

const kidneysInput = z.literal("1").or(z.literal("2"));

app.post("/health-checkup", function (req, res) {
  // do something with kidney here
  const kidneyId = req.body.kidneyId;
  const validation = kidneysInput.safeParse(kidneyId)
  if (!validation.success) {
    res.send("Incorrect input");
    return;
  }
  res.send("Your kidney id is " + kidneyId);
});


app.listen(3000);
```

# How can you do better input validation?

```ts
import { z } from "zod";

// primitive values
z.string();
z.number();
z.bigint();
z.boolean();
z.date();
z.symbol();

// empty types
z.undefined();
z.null();
z.void(); // accepts undefined

// catch-all types
// allows any value
z.any();
z.unknown();

// never type
// allows no values
z.never();
```

## Coercion for primitives

Zod now provides a more convenient way to coerce primitive values.

```ts
const schema = z.coerce.string();
schema.parse("tuna"); // => "tuna"
schema.parse(12); // => "12"
schema.parse(true); // => "true"
```

https://zod.dev/?id=basic-usage

# 3.1
# Middlewares, authentication, global catches, zod

# Authentication

**As you can tell by now, anyone can send requests to your backend**
**They can just go to postman and send a request**
**How do you ensure that this user has access to a certain resource?**

# Authentication

**Dumb way - Ask user to send username and password in all requests as headers**

# Authentication

**Slightly better way -**

1. **Give the user back a token on signup/signin**
2. **Ask the user to send back the token in all future requests**
3. **When the user logs out, ask the user to forget the token (or revoke it from the backend)**

# Authentication

**Library we need to get comfortable in - jsonwebtokens**